

To access or connect to a database, we can use the *psql* SQL prompt. To connect to the database, execute the following command:

```
$ psql -U postgres
```

To list all available DBs, execute the following command in the *psql* prompt:

```
postgres=# \l
               List of databases
Name | Owner | Encoding | Collate | Ctype | Access privileges
-----+-----+-----+-----+-----+-----
testdb |      | UTF8     | en_IN  | en_IN |
```

This command lists our 'testdb' database. We can create multiple databases according to our requirements. To perform any operation on the database, we first have to establish a connection with it. Execute the command given below with the *psql* prompt to do this:

```
postgres=# \c testdb
```

Upon successful connection, the following messages will be shown in the terminal:

```
You are now connected to database "testdb" as user
"postgres". Also prompt will change to 'testdb=#'.
```

To delete a database, we can use the DROP DATABASE statement as follows:

```
postgres=# DROP DATABASE testdb;
```



Note: Please ensure that each SQL query is terminated by a semicolon when using the *psql* prompt

PostgreSQL data types

Along with basic data types PostgreSQL supports many advanced data types. In this section, we'll discuss some of the important ones.

- 1) **Boolean:** Valid value for this data type is *true* or *false*.
- 2) **Character:** This data type is further divided into three sub-categories:
 - a) **CHAR(n):** This is a fixed-length data type. If you insert a string that is shorter than the length of the column, PostgreSQL pads spaces to it.
 - b) **VARCHAR(n):** This is a variable-length data type. PostgreSQL does not pad spaces when the stored string is shorter than the length of the column.
 - c) **TEXT:** This is also a variable-length data type. Theoretically, text data is a character string with unlimited length.

- 3) **Numeric:** This data type is further divided into two sub-categories:
 - a) **Integer:** Integers can be 2- or 4-byte long. 2-byte integers are denoted by SMALLINT and the valid range is -32768 to 32767. 4-byte integers are denoted by INT and the valid range is -214783648 to +214783647.
 - b) **Floating point number:** Single precision floating point numbers are denoted by *float*, whereas double precision floating point numbers are denoted by *real* or *float8*.
- 4) **Temporal data types:** Temporal data types allow you to store date and time. PostgreSQL supports the following temporal data types:
 - a) **DATE:** Stores date
 - b) **TIME:** Stores time
 - c) **TIMESTAMP:** Stores date and time
 - d) **TIMESTAMPZ:** Stores both timestamp and time zone data
 - e) **INTERVAL:** Stores periods of time
- 5) **Arrays:** PostgreSQL can store arrays of strings, integers and so on.
- 6) **JSON:** This data type is further divided into two sub-categories:
 - a) **JSON:** This is plain JSON and requires re-parsing for each operation.
 - b) **JSONB:** This stores JSON data in binary format. Unlike JSON, it supports indexing.
- 7) **UUID:** This data type allows you to store Universal Unique Identifiers defined by RFC-4122.
- 8) **Special data types:** Besides the primitive data types, PostgreSQL also provides several special geometric and network related data types as listed below:
 - a) **box:** A rectangular box
 - b) **line:** A set of points
 - c) **point:** A geometric pair of numbers
 - d) **lseg:** A line segment
 - e) **polygon:** A closed geometric
 - f) **inet:** IP4 address
 - g) **macaddr:** A MAC address

SQL queries in PostgreSQL

In this section, we'll discuss CRUD operations on tables. We'll discuss various SQL statements with relevant examples. SQL is a case-insensitive language. However, in this article, all SQL keywords are denoted using uppercase letters.

Managing tables

We can use the **CREATE TABLE** statement to create a new table. Its syntax is as follows:

```
CREATE TABLE table_name (
    column_name1 column_data_type,
    column_name2 column_data_type,
```